

Dendê Softhouse: Regras de Associação

Autor: Antônio Gabriel Gomes Falcão do Nascimento
Giovanna Oliveira Araujo
Gustavo Costa Pereira
João Magalhães Mira Neto

Agenda

O objetivo desta apresentação é explicar o algoritmo de regras de associação “Apriori”, utilizado como técnica de mineração de dados, bem como as suas etapas e aplicação.

1. Contextualização

Exibição dos processos anteriores e próximos de desenvolvimento do desafio do Dendê Softhouse.

2. Explicação do Algoritmo de Regra de Associação

Explicação da regra de Antecedente -> Consequente, Suporte, Confiança e Lift.

3. Abordagem do Algoritmo Apriori:

Características principais do Apriori e exemplo de execução do algoritmo para um pequeno conjunto de dados.

4. Aplicação no Dataset:

Abordagens das etapas iniciais, definição de Suporte e Confiança e regras encontradas.

1. Contextualização

Modelo de Recomendação - Programa de Trainee da Dendê Softhouse		
Etapas		
1	Análise Exploratória	Modelo de aplicação para análise
2	Pré-Processamento	Limpeza de dados
3	Algoritmos de Regra de Associação	Descoberta de padrões de consumo e afinidade
4	Modelo de Predição	Descoberta de padrões de consumo e afinidade

2. Explicação do Algoritmo de Regra de Associação

Um algoritmo de regra de associação é uma técnica de mineração de dados que tem como finalidade identificar relações entre variáveis numa base de dados

Uma regra de associação é composta por duas partes principais:

o Antecedente e o Consequente



2 . Explicação do Algoritmo de Regra de Associação

Estrutura da Regra: A -> B

Antecedente (A): É a condição, item ou conjunto de itens que é testado (o "se").

Consequente (B): É o resultado que ocorre caso a condição do antecedente for atendida (o "então").



2 . Explicação do Algoritmo de Regra de Associação



O Suporte é a métrica que mede a popularidade de uma classe dentro de uma base de dados, a frequência relativa dela

$$\text{Sup}(X) = \frac{(X \text{ in } D)}{(D)} \quad \leftarrow \text{Total de Transações}$$

↑

Transações contendo X(item)

2 . Explicação do Algoritmo de Regra de Associação

A confiança mede a probabilidade do item B ser comprado dado que o Item A já está na cesta de compras.



$$\text{Confiança} = \text{Suporte tem A e B} / \text{Suporte Item A}$$

2. Explicação do Algoritmo de Regra de Associação



O Lift é o responsável por separar uma associação real de uma coincidência, por exemplo:
O Item A realmente aumenta a chance de venda do Item B, ou o Item B vende bem sozinho de qualquer jeito?

$$\text{Lift} = \text{Confiança} / \text{Suporte_item_B}$$



3. Abordagem do Algoritmo Apriori

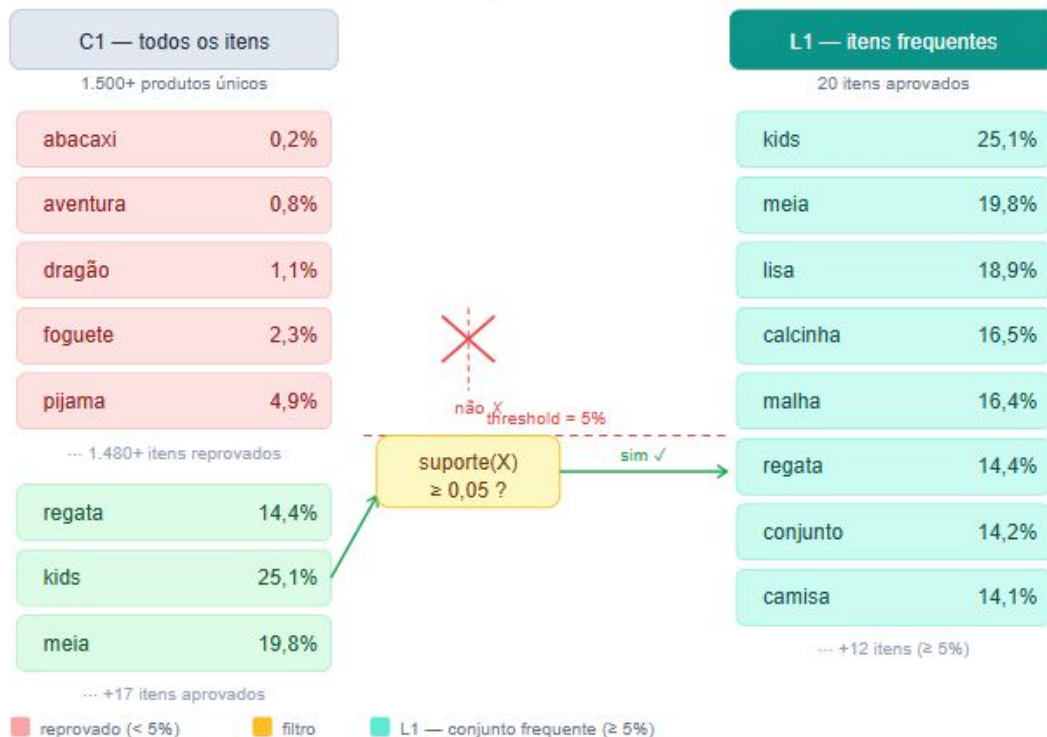
É um algoritmo de busca iterativa que utiliza um processo de "poda" (pruning) baseado no princípio de que: se um conjunto de itens é frequente, todos os seus subconjuntos também devem ser frequentes. Entre suas características estão:

Bottom-Up: Ele começa analisando itens individuais, depois sobe para pares e assim por diante.

Princípio de Monotonicidade: É a regra lógica que permite ao algoritmo descartar candidatos sem precisar ler a base de dados inteira várias vezes. Se "Meia" não vende, "Meia + Tênis" nunca será um combo frequente.

Geração de Regras: Após identificar os itens que aparecem muito (frequentes), o algoritmo calcula a Confiança e o Lift para transformar esses dados em regras de negócio (ex: "Quem compra X, tem 80% de chance de comprar Y").

3. Abordagem do Algoritmo Apriori



4. Aplicação no Dataset

1. Pré-processamento para Regras de Associação

Objetivo: Preparar os dados de transações para aplicação do algoritmo Apriori.

- Limpeza textual
- Padronização dos dados
- Remoção de ruídos
- Transformação em dados binários



4. Aplicação no Dataset

2. Tratamento das Descrições

O que foi removido?

1. Caracteres especiais;
2. Espaços extras;
3. Diferenças entre maiúsculas e minúsculas.

```
df["descricao_produtos"] = df["descricao_produtos"].str.strip().str.lower()
```

```
df["descricao_produtos"] = df["descricao_produtos"].str.replace(r'^\w\s+', ' ', regex=True)
```

4. Aplicação no Dataset

Remoção de Termos Irrelevantes

- Regras aplicadas:
 1. Remoção de stopwords;
 2. Remoção de números;
 3. Remoção de palavras muito curtas;
 4. Remoção de medidas técnicas.



4. Aplicação no Dataset

4. Estruturação dos Dados

- Separação dos produtos

```
itens = str(products).split(";")
```

- Remoção de duplicatas

```
cleaned_transaction = list(set(cleaned_transaction))
```

ID DA TRANSAÇÃO	PRODUTOS ADQUIRIDOS
1	camiseta , tenis, meia, boné
2	bermuda, camiseta, chinelo
3	jaqueta , calça, cinto, mochila
4	tenis, camiseta, meias (pack 3)
5	bermuda, regata, óculos de sol
6	vestido , sandália, colar
7	camiseta, bermuda, relógio
8	tênis esportivo, short, garrafa d'água
9	jaqueta de couro, calça jeans
10	sapato casual , meia calça, cinto

4. Aplicação no Dataset

Binarização do Dataset:

- Representação da presença ou ausência dos itens através de zeros e uns

```
df_binarizado = transactions_df["transactions"].str.get_dummies(sep=";")
```

TABELA BINÁRIA DE TRANSAÇÕES DE PRODUTOS

ID Transação	 Camiseta	 Tênis	 Bermuda
T01	1	1	0
T02	0	1	1
T03	1	0	1
T04	1	1	1

0 = Ausente | 1 = Presente (Verde)

4. Aplicação no Dataset

- *Primeira Etapa: Regra de Associação (Antecedente -> Consequente)*

Antecedente	O que o cliente já tem no carrinho. É a "condição". Se isso aconteceu...
Consequente	O que ele tende a levar junto. É a "conclusão". ...então isso provavelmente também aconteceu.

Exemplo:

{kids} -> {Meia}

Quem comprou kids provavelmente comprou meia

4. Aplicação no Dataset

- Segunda Etapa: **Implementação da contagem de frequência de itens individuais**

Separando e armazenando

C1 Quantidade de transações em cada produto, individualmente

Aplicando fórmula do suporte

Suporte	$\text{Sup}(X) = (X \text{ in } D) / (D)$
----------------	---

itemset Transação contendo X Total de transações

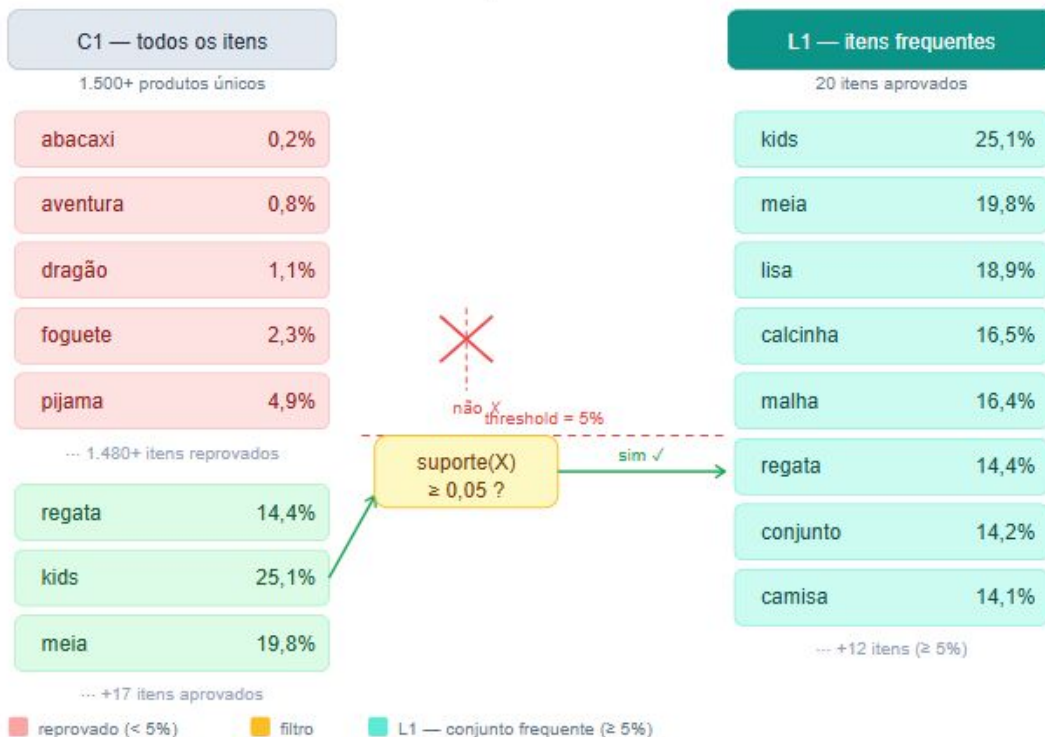
```
def calcular_suporte(self, item):  
    return self.df[item].mean()
```

4. Aplicação no Dataset

*Itens acima de 5% de suporte
-> Entram para o conjunto
frente (L1)*

L1 Acima do suporte

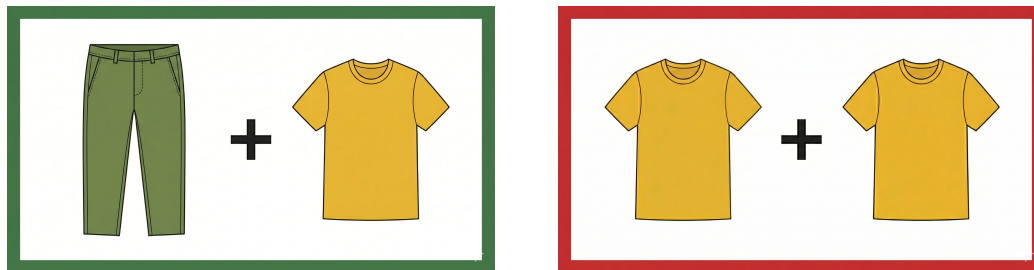
```
def obter_item_frequente(self):  
    itemset = {}  
    treshold = 0.05
```



4. Aplicação no Dataset

- Integração e Geração de Pares:

Usando a biblioteca “[intertools.combinations](#)”, é possível mapear todos os pares possíveis, eliminando pares repetidos e pares formados por elementos iguais.



4. Aplicação no Dataset

- Integração e Geração de Pares:

```
# Transforma as chaves do dicionário em uma lista de nomes
nomes_das_roupas = list(itens_frequentes.keys())

# Pega a lista de roupas e forma todos os pares possíveis (tamanho 2)
pares = list(combinations(nomes_das_roupas, 2))
# O for loop vai passar por cada par gerado para você calcular os suportes
for par in pares:
    item_a = par[0] # Pega a primeira peça do par
    item_b = par[1] # Pega a segunda peça do par
```

4. Aplicação no Dataset

- Aplicando o Filtro de Suporte:

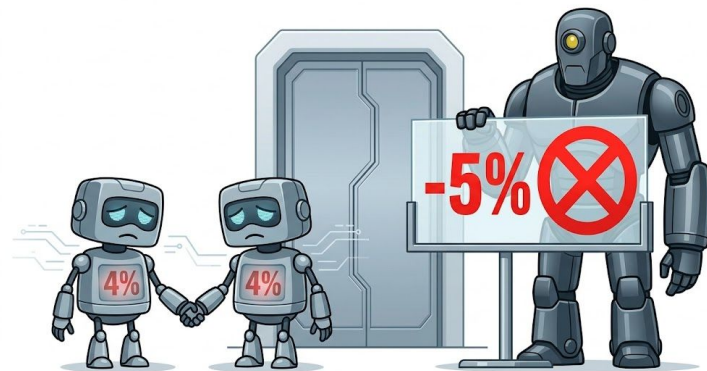
Para que o código avance, os pares precisam aparecer juntos em no mínimo 5% das vendas totais da empresa.

```
for par in pares:
    item_a = par[0] # Pega a primeira peça do par
    item_b = par[1] # Pega a segunda peça do par

    suporte_combo = (dados[item_a] * dados[item_b]).mean()

    if suporte_combo >= 0.05:

        suporte_a = itens_frequentes[item_a]
        suporte_b = itens_frequentes[item_b]
```



4. Aplicação no Dataset

- Aplicando os cálculos de confiança e Lift , printando o resultado

```
1  #Contas de confiança e Lift
2      confianca = suporte_combo / suporte_a
3      lift = confianca / suporte_b
4
5      #Guarda tudo no dicionario arredondando para 4 casas decimais
6      combos[par] = {
7          'Suporte': round(float(suporte_combo), 4),
8          'Confianca': round(float(confianca), 4),
9          'Lift': round(float(lift), 4)
10     }
11
12
13     print("combos interessantes para possivel campanha:", combos)
```

4. Aplicação no Dataset

- Resultados no CSV

	A	B	C	D	E
1	Item 1	Item 2	Suporte	Confianca	Lift
2	algodão	calcinha	0.0527	0.4161	2.5241
3	algodão	camiseta	0.0599	0.4731	5.7463
4	algodão	lisa	0.0657	0.519	2.7445
5	baby	infantil	0.0629	0.4486	3.398
6	baby	kids	0.0771	0.55	2.1912
7	baby	longo	0.0595	0.4243	4.9487
8	baby	micol	0.0673	0.48	6.1283
9	baby	pijama	0.0543	0.3871	4.112
10	calcinha	estampada	0.0547	0.3317	3.5998
11	calcinha	kids	0.0729	0.4423	1.7621
12	calcinha	lisa	0.0743	0.4508	2.3838

Referências

- NOBLE, Joshua. O que é o algoritmo Apriori? IBM. Disponível em: <https://www.ibm.com/br-pt/think/topics/apriori-algorithm>. Acesso em: 9 de maio de 2026.
- MWITI, Derrick. Apriori Algorithm Explained: A Step-by-Step Guide with Python Implementation. DataCamp, 15 abr. 2025. Disponível em: <https://www.datacamp.com/tutorial/apriori-algorithm>. Acesso em: 9 de maio de 2026.
- HAN, Jiawei; KAMBER, Micheline; PEI, Jian. Data mining: concepts and techniques. 3. ed. San Francisco: Morgan Kaufmann, 2011. Disponível em: <https://www.sciencedirect.com/book/monograph/9780123814791/data-mining-concepts-and-techniques>. Acesso em: 10 de maio 2026.